Module 25

Relational Database Design/5

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Studied Algorithms for Properties of Functional Dependencies

Module Objectives

Goals

- To Understand the Characterizations for Lossless Join Decomposition

Module Objectives

Goals

- To Understand the Characterizations for Lossless Join Decomposition
- To Understand the Characterizations for Dependency Preservation

Module Outline

- Lossless Join Decomposition

Module Outline

- Lossless Join Decomposition
- Dependency Preservation

Lossless Join Decomposition

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless join if at least one of the following dependencies is in F^+:

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless join if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless join if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless join if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies

Lossless Join Decomposition (continued)

To Identify whether a decomposition is lossy or lossless, it must satisfy the following conditions:

- $R_1 \cup R_2 = R$

Lossless Join Decomposition (continued)

To Identify whether a decomposition is lossy or lossless, it must satisfy the following conditions:

- $R_1 \cup R_2 = R$
- $R_1 \cap R_2 \neq \phi$ and

Lossless Join Decomposition (continued)

To Identify whether a decomposition is lossy or lossless, it must satisfy the following conditions:

- $R_1 \cup R_2 = R$
- $R_1 \cap R_2 \neq \phi$ and
- $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$

Lossless Join Decomposition (2): Example

Lossy Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`

Lossless Join Decomposition (2): Example

Lossy Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`

Lossless Join Decomposition (2): Example

Lossy Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City, Qty) : Parts(P#, Qty)`

Lossless Join Decomposition (2): Example

Lossy Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City, Qty)` : `Parts(P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`

Lossless Join Decomposition (2): Example

Lossy Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City, Qty)` : `Parts(P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`
- We get extra tuples! Join is **Lossy**!

Lossless Join Decomposition (2): Example

Lossy Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City, Qty)` : `Parts(P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`
- We get extra tuples! Join is **Lossy**!
- Common attribute `Qty` is not a superkey in Supplier or in Parts

Lossless Join Decomposition (2): Example

Lossy Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City, Qty)` : `Parts(P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`
- We get extra tuples! Join is **Lossy**!
- Common attribute `Qty` is not a superkey in Supplier or in Parts
- Does not preserve `(S#, P#)` $\rightarrow$ `Qty`

Lossless Join Decomposition (3): Example

Lossless Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`

Lossless Join Decomposition (3): Example

Lossless Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`

Lossless Join Decomposition (3): Example

Lossless Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City) : Parts(S#, P#, Qty)`

Lossless Join Decomposition (3): Example

Lossless Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City)` : `Parts(S#, P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`

Lossless Join Decomposition (3): Example

Lossless Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City)` : `Parts(S#, P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`
- We get back the original relation. Join is **Lossless**.

Lossless Join Decomposition (3): Example

Lossless Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City)` : `Parts(S#, P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`
- We get back the original relation. Join is **Lossless**.
- Common attribute `S#` is a superkey in Supplier

Lossless Join Decomposition (3): Example

Lossless Decomposition Example

- Consider Supplier Parts schema: `Supplier_Parts(S#, Sname, City, P#, Qty)`
- Having dependencies: `S#` $\rightarrow$ `Sname`, `S#` $\rightarrow$ `City`, `(S#, P#)` $\rightarrow$ `Qty`
- Decompose as: `Supplier(S#, Sname, City)` : `Parts(S#, P#, Qty)`
- Take Natural Join to reconstruct: `Supplier` $\bowtie$ `Parts`
- We get back the original relation. Join is **Lossless**.
- Common attribute `S#` is a superkey in Supplier
- Preserves all dependencies

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B), R_2 = (B, C)$

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B), R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B), R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B), R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = (A, B), R_2 = (A, C)$

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B), R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = (A, B), R_2 = (A, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$

Lossless Join Decomposition (4): Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B), R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = (A, B), R_2 = (A, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
  - Not dependency preserving (cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \rightarrow B, A \rightarrow C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, E \rightarrow F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$
- d) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$
- d) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$
  - i) $D_1 = R_1(AB), R_2(BC), R_3(ABDE), R_4(EG)$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$
- d) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$
  - i) $D_1 = R_1(AB), R_2(BC), R_3(ABDE), R_4(EG)$
  - ii) $D_2 = R_1(ABC), R_2(ACDE), R_3(ADG)$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$
- d) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$
  - i) $D_1 = R_1(AB), R_2(BC), R_3(ABDE), R_4(EG)$
  - ii) $D_2 = R_1(ABC), R_2(ACDE), R_3(ADG)$
- e) $R(ABCDEFGHIJ)$: $F = \{AB \to C, B \to F, D \to IJ, A \to DE, F \to GH\}$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$
- d) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$
  - i) $D_1 = R_1(AB), R_2(BC), R_3(ABDE), R_4(EG)$
  - ii) $D_2 = R_1(ABC), R_2(ACDE), R_3(ADG)$
- e) $R(ABCDEFGHIJ)$: $F = \{AB \to C, B \to F, D \to IJ, A \to DE, F \to GH\}$
  - i) $D_1 = R_1(ABC), R_2(ADE), R_3(BF), R_4(FGH), R_5(DIJ)$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$
- d) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$
  - i) $D_1 = R_1(AB), R_2(BC), R_3(ABDE), R_4(EG)$
  - ii) $D_2 = R_1(ABC), R_2(ACDE), R_3(ADG)$
- e) $R(ABCDEFGHIJ)$: $F = \{AB \to C, B \to F, D \to IJ, A \to DE, F \to GH\}$
  - i) $D_1 = R_1(ABC), R_2(ADE), R_3(BF), R_4(FGH), R_5(DIJ)$
  - ii) $D_2 = R_1(ABCDE), R_2(BFGH), R_3(DIJ)$

Practice Problems on Lossless Join

- Check if the decomposition of R into D is lossless:
- a) $R(ABC)$: $F = \{A \to B, A \to C\}$. $D = R_1(AB), R_2(BC)$
- b) $R(ABCDEF)$: $F = \{A \to B, B \to C, C \to D, E \to F\}$.
  $D = R_1(AB), R_2(BCD), R_3(DEF)$
- c) $R(ABCDEF)$: $F = \{A \to B, C \to DE, AC \to F\}$. $D = R_1(BE), R_2(ACDEF)$
- d) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$
  - i) $D_1 = R_1(AB), R_2(BC), R_3(ABDE), R_4(EG)$
  - ii) $D_2 = R_1(ABC), R_2(ACDE), R_3(ADG)$
- e) $R(ABCDEFGHIJ)$: $F = \{AB \to C, B \to F, D \to IJ, A \to DE, F \to GH\}$
  - i) $D_1 = R_1(ABC), R_2(ADE), R_3(BF), R_4(FGH), R_5(DIJ)$
  - ii) $D_2 = R_1(ABCDE), R_2(BFGH), R_3(DIJ)$
  - iii) $D_3 = R_1(ABCD), R_2(DE), R_3(BF), R_4(FGH), R_5(DIJ)$

Dependency Preservation

Dependency Preservation

Definition

- Let F_i be the set of dependencies F^+ that include only attributes in R_i

Dependency Preservation

Definition

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

Dependency Preservation

Definition

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

- If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive

Dependency Preservation (continued)

Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if

- $F_1 \cup F_2 \equiv F$ { Decomposition Preserving Dependency }

Dependency Preservation (continued)

Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if

- $F_1 \cup F_2 \equiv F$ { Decomposition Preserving Dependency }
- If $F_1 \cup F_2 \subset F$ { Decomposition NOT Preserving Dependency } and

Dependency Preservation (continued)

Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if

- $F_1 \cup F_2 \equiv F$ { Decomposition Preserving Dependency }
- If $F_1 \cup F_2 \subset F$ { Decomposition NOT Preserving Dependency } and
- $F_1 \cup F_2 \supset F$ { this is not possible }

Dependency Preservation (2): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $D = \{R_1, R_2, \ldots, R_n\}$ we apply the following test (with attribute closure done with respect to F)

```
compute F+;
for each schema Ri in D do begin
    Fi = the restriction of F+ to Ri;
end
F' = phi
for each restriction Fi do begin
    F' = F' union Fi
end
compute F'+;
if (F'+ == F+) then return (true) else return (false);
```

Dependency Preservation (2): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $D = \{R_1, R_2, \ldots, R_n\}$ we apply the following test (with attribute closure done with respect to F)
- The restriction of F^+ to R_i is the set of all functional dependencies in F^+ that include only attributes of R_i.

```
compute F+;
for each schema Ri in D do begin
    Fi = the restriction of F+ to Ri;
end
F' = phi
for each restriction Fi do begin
    F' = F' union Fi
end
compute F'+;
if (F'+ == F+) then return (true) else return (false);
```

Dependency Preservation (2): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $D = \{R_1, R_2, \ldots, R_n\}$ we apply the following test (with attribute closure done with respect to F)
- The restriction of F^+ to R_i is the set of all functional dependencies in F^+ that include only attributes of R_i.

```
compute F+;
for each schema Ri in D do begin
    Fi = the restriction of F+ to Ri;
end
F' = phi
for each restriction Fi do begin
    F' = F' union Fi
end
compute F'+;
if (F'+ == F+) then return (true) else return (false);
```

Dependency Preservation (2): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $D = \{R_1, R_2, \ldots, R_n\}$ we apply the following test (with attribute closure done with respect to F)
- The restriction of F^+ to R_i is the set of all functional dependencies in F^+ that include only attributes of R_i.

```
compute F+;
for each schema Ri in D do begin
    Fi = the restriction of F+ to Ri;
end
F' = phi
for each restriction Fi do begin
    F' = F' union Fi
end
compute F'+;
if (F'+ == F+) then return (true) else return (false);
```

Dependency Preservation (2): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $D = \{R_1, R_2, \ldots, R_n\}$ we apply the following test (with attribute closure done with respect to F)
- The restriction of F^+ to R_i is the set of all functional dependencies in F^+ that include only attributes of R_i.

```
compute F+;
for each schema Ri in D do begin
    Fi = the restriction of F+ to Ri;
end
F' = phi
for each restriction Fi do begin
    F' = F' union Fi
end
compute F'+;
if (F'+ == F+) then return (true) else return (false);
```

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$
- $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$
- $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$
- Decomposition:

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$
- $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$
- Decomposition:
- $R_1(A, B, C, D) \qquad R_2(B, F) \qquad R_3(D, E)$

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$
- $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomposition:
- $R_1(A, B, C, D) \qquad R_2(B, F) \qquad R_3(D, E)$
- $A \to BCD, BC \to AD$ are preserved on table R_1

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$
- $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$
- Decomposition:
- $R_1(A, B, C, D) \qquad R_2(B, F) \qquad R_3(D, E)$
- $A \rightarrow BCD, BC \rightarrow AD$ are preserved on table R_1
- $D \rightarrow E$ is preserved on table R_3

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$
- $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$
- Decomposition:
- $R_1(A, B, C, D) \qquad R_2(B, F) \qquad R_3(D, E)$
- $A \rightarrow BCD, BC \rightarrow AD$ are preserved on table R_1
- $D \rightarrow E$ is preserved on table R_3
- $B \rightarrow F$ is preserved on table R_2

Dependency Preservation (3): Example

- $R(A, B, C, D, E, F)$
- $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$
- Decomposition:
- $R_1(A, B, C, D) \qquad R_2(B, F) \qquad R_3(D, E)$
- $A \rightarrow BCD, BC \rightarrow AD$ are preserved on table R_1
- $D \rightarrow E$ is preserved on table R_3
- $B \rightarrow F$ is preserved on table R_2
- We have to check whether the remaining FDs: $A \rightarrow E, A \rightarrow F, BC \rightarrow E, BC \rightarrow F$ are preserved or not.

Dependency Preservation (3): Example - Checking

- $F' = F_1 \cup F_2 \cup F_3$

Dependency Preservation (3): Example - Checking

- $F' = F_1 \cup F_2 \cup F_3$
- Checking for: $A \rightarrow E, A \rightarrow F$ in F'^{+}

Dependency Preservation (3): Example - Checking

- $F' = F_1 \cup F_2 \cup F_3$
- Checking for: $A \rightarrow E, A \rightarrow F$ in F'^{+}
  - $A \rightarrow D$ (from R_1), $D \rightarrow E$ (from R_3) : $A \rightarrow E$ (By Transitivity)

Dependency Preservation (3): Example - Checking

- $F' = F_1 \cup F_2 \cup F_3$
- Checking for: $A \rightarrow E, A \rightarrow F$ in F'^{+}
  - $A \rightarrow D$ (from R_1), $D \rightarrow E$ (from R_3) : $A \rightarrow E$ (By Transitivity)
  - $A \rightarrow B$ (from R_1), $B \rightarrow F$ (from R_2) : $A \rightarrow F$ (By Transitivity)

Dependency Preservation (3): Example - Checking

- $F' = F_1 \cup F_2 \cup F_3$
- Checking for: $A \rightarrow E, A \rightarrow F$ in F'^+
  - $A \rightarrow D$ (from R_1), $D \rightarrow E$ (from R_3) : $A \rightarrow E$ (By Transitivity)
  - $A \rightarrow B$ (from R_1), $B \rightarrow F$ (from R_2) : $A \rightarrow F$ (By Transitivity)
- Checking for: $BC \rightarrow E, BC \rightarrow F$ in F'^+

Hence all dependencies are preserved.

Dependency Preservation (3): Example - Checking

- $F' = F_1 \cup F_2 \cup F_3$
- Checking for: $A \rightarrow E, A \rightarrow F$ in F'^+
  - $A \rightarrow D$ (from R_1), $D \rightarrow E$ (from R_3) : $A \rightarrow E$ (By Transitivity)
  - $A \rightarrow B$ (from R_1), $B \rightarrow F$ (from R_2) : $A \rightarrow F$ (By Transitivity)
- Checking for: $BC \rightarrow E, BC \rightarrow F$ in F'^+
  - $BC \rightarrow D$ (from R_1), $D \rightarrow E$ (from R_3) : $BC \rightarrow E$ (By Transitivity)

Hence all dependencies are preserved.

Dependency Preservation (3): Example - Checking

- $F' = F_1 \cup F_2 \cup F_3$
- Checking for: $A \rightarrow E, A \rightarrow F$ in F'^+
  - $A \rightarrow D$ (from R_1), $D \rightarrow E$ (from R_3) : $A \rightarrow E$ (By Transitivity)
  - $A \rightarrow B$ (from R_1), $B \rightarrow F$ (from R_2) : $A \rightarrow F$ (By Transitivity)
- Checking for: $BC \rightarrow E, BC \rightarrow F$ in F'^+
  - $BC \rightarrow D$ (from R_1), $D \rightarrow E$ (from R_3) : $BC \rightarrow E$ (By Transitivity)
  - $BC \rightarrow F$ (By Augmentation on $B \rightarrow F$ from R_2)

Hence all dependencies are preserved.

Dependency Preservation (4): Example

- $R(A, B, C, D)$

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F' = F_1 \cup F_2 \cup F_3$.

Hence all dependencies are preserved.

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F' = F_1 \cup F_2 \cup F_3$.
- Checking for: $D \rightarrow A$ in F'^{+}

Hence all dependencies are preserved.

Dependency Preservation (4): Example

- $R(A, B, C, D)$
- $F = \{A \to B, B \to C, C \to D, D \to A\}$
- Decomposition:
- $R_1(A, B) \qquad R_2(B, C) \qquad R_3(C, D)$
- $A \to B$ is preserved on table R_1
- $B \to C$ is preserved on table R_2
- $C \to D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \to A$ is preserved or not.
- $F' = F_1 \cup F_2 \cup F_3$.
- Checking for: $D \to A$ in F'^{+}
  - $D \to C$ (from R_3), $C \to B$ (from R_2), $B \to A$ (from R_1) : $D \to A$ (By Transitivity)

Hence all dependencies are preserved.

Dependency Preservation (5): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \ldots, R_n$ we apply the following test (with attribute closure done with respect to F)

```
result = alpha
while (changes to result) do
    for each R_i in the decomposition
        t = (result intersect R_i)^+ intersect R_i
        result = result union t
```

Dependency Preservation (5): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \ldots, R_n$ we apply the following test (with attribute closure done with respect to F)

```
result = alpha
while (changes to result) do
    for each R_i in the decomposition
        t = (result intersect R_i)^+ intersect R_i
        result = result union t
```

Dependency Preservation (5): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \ldots, R_n$ we apply the following test (with attribute closure done with respect to F)

```
result = alpha
while (changes to result) do
    for each R_i in the decomposition
        t = (result intersect R_i)^+ intersect R_i
        result = result union t
```

Dependency Preservation (5): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \ldots, R_n$ we apply the following test (with attribute closure done with respect to F)

```
result = alpha
while (changes to result) do
    for each R_i in the decomposition
        t = (result intersect R_i)^+ intersect R_i
        result = result union t
```

- If `result` contains all attributes in β, then the functional dependency $\alpha \rightarrow \beta$ is preserved.

Dependency Preservation (5): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \ldots, R_n$ we apply the following test (with attribute closure done with respect to F)

```
result = alpha
while (changes to result) do
    for each R_i in the decomposition
        t = (result intersect R_i)^+ intersect R_i
        result = result union t
```

- If `result` contains all attributes in β, then the functional dependency $\alpha \rightarrow \beta$ is preserved.
- We apply the test on all dependencies in F to check if a decomposition is dependency preserving

Dependency Preservation (5): Testing

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \ldots, R_n$ we apply the following test (with attribute closure done with respect to F)

```
result = alpha
while (changes to result) do
    for each R_i in the decomposition
        t = (result intersect R_i)^+ intersect R_i
        result = result union t
```

- If `result` contains all attributes in β, then the functional dependency $\alpha \rightarrow \beta$ is preserved.
- We apply the test on all dependencies in F to check if a decomposition is dependency preserving
- This procedure takes polynomial time, instead of the exponential time required to compute F^+ and $(F_1 \cup F_2 \cup \cdots \cup F_n)^+$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$
- Need to check for: $\bigstar$ $A \to BCD, A \to EF$, $\bigstar$ $BC \to AD, BC \to E, BC \to F$, $\times$ $B \to F$, $\times$ $D \to E$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$
- Need to check for: ★ $A \to BCD, A \to EF$, ★ $BC \to AD, BC \to E, BC \to F$, × $B \to F$, × $D \to E$
- $(BC)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $BC \to E, BC \to F$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$
- Need to check for: ★ $A \to BCD, A \to EF$, ★ $BC \to AD, BC \to E, BC \to F$, × $B \to F$, × $D \to E$
- $(BC)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $BC \to E, BC \to F$
  - $BC \to AD(R_1), AD \to E(R_3)$ implies $BC \to E$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$
- Need to check for: ★ $A \to BCD, A \to EF$, ★ $BC \to AD, BC \to E, BC \to F$, × $B \to F$, × $D \to E$
- $(BC)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $BC \to E, BC \to F$
  - $BC \to AD(R_1), AD \to E(R_3)$ implies $BC \to E$
  - $B \to F(R_2)$ implies $BC \to F$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$
- Need to check for: ★ $A \to BCD, A \to EF$, ★ $BC \to AD, BC \to E, BC \to F$, × $B \to F$, × $D \to E$
- $(BC)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $BC \to E, BC \to F$
  - $BC \to AD(R_1), AD \to E(R_3)$ implies $BC \to E$
  - $B \to F(R_2)$ implies $BC \to F$
- $(A)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $A \to EF$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$
- Need to check for: ★ $A \to BCD, A \to EF$, ★ $BC \to AD, BC \to E, BC \to F$, × $B \to F$, × $D \to E$
- $(BC)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $BC \to E, BC \to F$
  - ▷ $BC \to AD(R_1), AD \to E(R_3)$ implies $BC \to E$
  - ▷ $B \to F(R_2)$ implies $BC \to F$
- $(A)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $A \to EF$
  - ▷ $A \to B(R_1), B \to F(R_2)$ implies $A \to F$

Dependency Preservation (6): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$
- Decomp $= \{ABCD, BF, DE\}$
- Need to check for: ★ $A \to BCD, A \to EF$, ★ $BC \to AD, BC \to E, BC \to F$, × $B \to F$, × $D \to E$
- $(BC)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $BC \to E, BC \to F$
  - $BC \to AD(R_1), AD \to E(R_3)$ implies $BC \to E$
  - $B \to F(R_2)$ implies $BC \to F$
- $(A)^+/F_1 = ABCD$. $(ABCD)^+/F_2 = ABCDF$. $(ABCDF)^+/F_3 = ABCDEF$. **Preserves** $A \to EF$
  - $A \to B(R_1), B \to F(R_2)$ implies $A \to F$
  - $A \to D(R_1), D \to E(R_3)$ implies $A \to E$

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$. Decomp $= \{ABCD, BF, DE\}$

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \rightarrow A$ cannot be inferred

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \rightarrow BCD, A \rightarrow EF, BC \rightarrow AD, BC \rightarrow E, BC \rightarrow F, B \rightarrow F, D \rightarrow E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \rightarrow A$ cannot be inferred
  - $C^+/F = C : C \rightarrow A$ cannot be inferred

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \to A$ cannot be inferred
  - $C^+/F = C : C \to A$ cannot be inferred
  - $D^+/F = DE : D \to A$ and $D \to BC$ cannot be inferred

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \to A$ cannot be inferred
  - $C^+/F = C : C \to A$ cannot be inferred
  - $D^+/F = DE : D \to A$ and $D \to BC$ cannot be inferred
  - $A^+/F = ABCDEF : A \to BC$ can be inferred, but it is equal to $A \to B$ and $A \to C$

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \to A$ cannot be inferred
  - $C^+/F = C : C \to A$ cannot be inferred
  - $D^+/F = DE : D \to A$ and $D \to BC$ cannot be inferred
  - $A^+/F = ABCDEF : A \to BC$ can be inferred, but it is equal to $A \to B$ and $A \to C$
  - $F^+/F = F : F \to B$ cannot be inferred

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \to A$ cannot be inferred
  - $C^+/F = C : C \to A$ cannot be inferred
  - $D^+/F = DE : D \to A$ and $D \to BC$ cannot be inferred
  - $A^+/F = ABCDEF : A \to BC$ can be inferred, but it is equal to $A \to B$ and $A \to C$
  - $F^+/F = F : F \to B$ cannot be inferred
  - $E^+/F = E : E \to D$ cannot be inferred

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \to A$ cannot be inferred
  - $C^+/F = C : C \to A$ cannot be inferred
  - $D^+/F = DE : D \to A$ and $D \to BC$ cannot be inferred
  - $A^+/F = ABCDEF : A \to BC$ can be inferred, but it is equal to $A \to B$ and $A \to C$
  - $F^+/F = F : F \to B$ cannot be inferred
  - $E^+/F = E : E \to D$ cannot be inferred
- $(BC)^+/F = ABCDEF$. **Preserves** $BC \to E, BC \to F$

Dependency Preservation (7): Example

- $R(ABCDEF)$:
  $F = \{A \to BCD, A \to EF, BC \to AD, BC \to E, BC \to F, B \to F, D \to E\}$. Decomp $= \{ABCD, BF, DE\}$
- Infer reverse FD's:
  - $B^+/F = BF : B \to A$ cannot be inferred
  - $C^+/F = C : C \to A$ cannot be inferred
  - $D^+/F = DE : D \to A$ and $D \to BC$ cannot be inferred
  - $A^+/F = ABCDEF : A \to BC$ can be inferred, but it is equal to $A \to B$ and $A \to C$
  - $F^+/F = F : F \to B$ cannot be inferred
  - $E^+/F = E : E \to D$ cannot be inferred
- $(BC)^+/F = ABCDEF$. **Preserves** $BC \to E, BC \to F$
- $(A)^+/F = ABCDEF$. **Preserves** $A \to EF$

Practice Problems on Dependency Preservation

- Check whether the decomposition of R into D is preserving dependency:

Practice Problems on Dependency Preservation

- Check whether the decomposition of R into D is preserving dependency:
- a) $R(ABCD)$: $F = \{A \to B, B \to C, C \to D, D \to A\}$. $D = \{AB, BC, CD\}$

Practice Problems on Dependency Preservation

- Check whether the decomposition of R into D is preserving dependency:
- a) $R(ABCD)$: $F = \{A \to B, B \to C, C \to D, D \to A\}$. $D = \{AB, BC, CD\}$
- b) $R(ABCDEF)$: $F = \{AB \to CD, C \to D, D \to E, E \to F\}$. $D = \{AB, CDE, EF\}$

Practice Problems on Dependency Preservation

- Check whether the decomposition of R into D is preserving dependency:
- a) $R(ABCD)$: $F = \{A \to B, B \to C, C \to D, D \to A\}$. $D = \{AB, BC, CD\}$
- b) $R(ABCDEF)$: $F = \{AB \to CD, C \to D, D \to E, E \to F\}$. $D = \{AB, CDE, EF\}$
- c) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, BC \to A, AD \to E, B \to D, E \to G\}$. $D = \{ABC, ACDE, ADG\}$

Practice Problems on Dependency Preservation

- Check whether the decomposition of R into D is preserving dependency:
- a) $R(ABCD)$: $F = \{A \to B, B \to C, C \to D, D \to A\}$. $D = \{AB, BC, CD\}$
- b) $R(ABCDEF)$: $F = \{AB \to CD, C \to D, D \to E, E \to F\}$. $D = \{AB, CDE, EF\}$
- c) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, BC \to A, AD \to E, B \to D, E \to G\}$. $D = \{ABC, ACDE, ADG\}$
- d) $R(ABCD)$: $F = \{A \to B, B \to C, C \to D, D \to B\}$. $D = \{AB, BC, BD\}$

Practice Problems on Dependency Preservation

- Check whether the decomposition of R into D is preserving dependency:
- a) $R(ABCD)$: $F = \{A \to B, B \to C, C \to D, D \to A\}$. $D = \{AB, BC, CD\}$
- b) $R(ABCDEF)$: $F = \{AB \to CD, C \to D, D \to E, E \to F\}$. $D = \{AB, CDE, EF\}$
- c) $R(ABCDEG)$: $F = \{AB \to C, AC \to B, BC \to A, AD \to E, B \to D, E \to G\}$. $D = \{ABC, ACDE, ADG\}$
- d) $R(ABCD)$: $F = \{A \to B, B \to C, C \to D, D \to B\}$. $D = \{AB, BC, BD\}$
- e) $R(ABCDE)$: $F = \{A \to BC, CD \to E, B \to D, E \to A\}$. $D = \{ABCE, BD\}$

Module Summary

Summary

- Understood the Characterization for and Determination of Lossless Join

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Understood the Characterization for and Determination of Lossless Join
- Understood the Characterization for and Determination of Dependency Preservation

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.